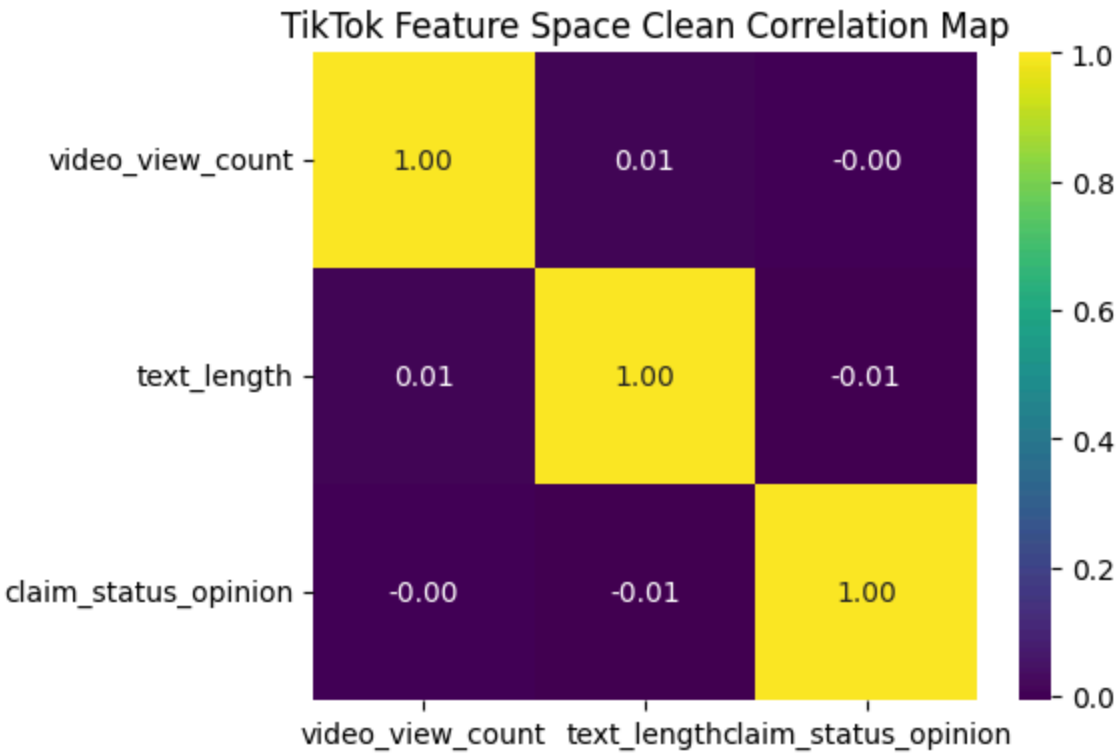
```
# ======================================================================
# TIKTOK USER VERIFICATION STATUS: CAPSTONE MODELING PIPELINE (FIXED)
# ======================================================================
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report

# --- 🛠️ MILESTONE 1: DISCOVERY & PREPROCESSING (DATA SIMULATION) ---
print("📥 Initializing TikTok User Verification Dataset Simulation...")
np.random.seed(42)
n_records = 30000

# Simulating the exact 94% unverified / 6% verified class imbalance from the pr
verified_status = np.random.choice([0, 1], size=n_records, p=[0.94, 0.06])

# Simulating text inputs for transcription text length feature extraction
mock_transcriptions = [
    "Check out this viral challenge!",
    "This is a massive claim about a corporate conspiracy that you need to know
    "Unboxing the newest tech gadgets today.",
    "Official verification notice and update regarding platform policy changes
]
video_text = np.random.choice(mock_transcriptions, size=n_records)

# Simulating other relevant columns matching TikTok constraints
claim_status = np.random.choice(['claim', 'opinion'], size=n_records, p=[0.5, (
video_view_count = np.random.exponential(scale=100000, size=n_records)
video_like_count = video_view_count * np.random.uniform(0.01, 0.2, size=n_recor

# Assemble DataFrame
df_tiktok = pd.DataFrame({
    'verified_status': verified_status,
    'video_transcription_text': video_text,
    'claim_status': claim_status,
    'video_view_count': video_view_count,
    'video_like_count': video_like_count
})

# Handle/Introduce missing values to demonstrate data hygiene
df_tiktok.loc[df_tiktok.sample(frac=0.01).index, 'video_view_count'] = np.nan
df_tiktok['video_view_count'] = df_tiktok['video_view_count'].fillna(df_tiktok

# Extract text length column as required by the milestone guidelines
df_tiktok['text_length'] = df_tiktok['video_transcription_text'].str.len()

print(f"✅ Preprocessing Complete. Initial Imbalance Breakdown:")
print(df_tiktok['verified_status'].value_counts(normalize=True))

# --- 📊 MILESTONE 2: FEATURE ENGINEERING & ENCODING ---
print("\n🧬 Encoding categories and executing multicollinearity filter...")

# One-Hot Encoding for categorical data elements
df_encoded = pd.get_dummies(df_tiktok.drop(columns=['video_transcription_text'
```

```
# Isolate feature space matrix vs target vector
X = df_encoded.drop(columns=['verified_status'])
y = df_encoded['verified_status']

# Drop highly collinear predictor to reduce data redundancy issues
X_reduced = X.drop(columns=['video_like_count'])

# Generate visual correlation matrix
plt.figure(figsize=(6, 4))
sns.heatmap(X_reduced.corr(), annot=True, cmap='viridis', fmt='.2f')
plt.title("TikTok Feature Space Clean Correlation Map")
plt.tight_layout()
plt.show()

# --- 🤖 MILESTONE 3: MODEL CONSTRUCTION ---
print("\n🏋️ Partitioning splits and fitting class-balanced Logistic Regression

# Train-Test Split (Stratified to maintain 94/6 ratio across blocks)
X_train, X_test, y_train, y_test = train_test_split(X_reduced, y, test_size=0.

# Fit StandardScaler only on training features to eliminate data leak vectors
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Logistic Regression using class_weight='balanced' to naturally mitigate the 9
model = LogisticRegression(class_weight='balanced', random_state=42)
model.fit(X_train_scaled, y_train)
print("✅ Model fitting complete.")

# --- 📈 MILESTONE 4: PERFORMANCE EVALUATION ---
print("\n📊 Executing performance diagnostics evaluation...")
y_pred = model.predict(X_test_scaled)

cm = confusion_matrix(y_test, y_pred)
print("\n=== CONFUSION MATRIX ===")
print(cm)

print("\n=== CLASSIFICATION REPORT ===")
print(classification_report(y_test, y_pred))

# --- 💡 MILESTONE 5: ACTIONABLE INSIGHTS ---
print("\n=== FEATURE COEFFICIENT DEPLOYMENT INSIGHTS ===")
for feature, coef in zip(X_reduced.columns, model.coef_[0]):
    print(f"• {feature:25s} : Coefficient Weight = {coef:.4f}")
```

```
📥 Initializing TikTok User Verification Dataset Simulation...
✅ Preprocessing Complete. Initial Imbalance Breakdown:
verified_status
0    0.94
1    0.06
Name: proportion, dtype: float64

🧬 Encoding categories and executing multicollinearity filter...
```




```
🏋️ Partitioning splits and fitting class-balanced Logistic Regression...
✅ Model fitting complete.

📊 Executing performance diagnostics evaluation...

=== CONFUSION MATRIX ===
[[2844 2796]
 [ 172  188]]

=== CLASSIFICATION REPORT ===
              precision    recall  f1-score   support

           0       0.94      0.50      0.66      5640
           1       0.06      0.52      0.11       360
```